

NS-3 Complete Tutorial Guide

From Zero to Simulation — Deep Dive into All 7 Examples

Table of Contents

1. What is NS-3?
2. Core Concepts
 - Node
 - Application
 - Channel
 - NetDevice
 - Topology Helpers
3. The Simulation Lifecycle
4. Logging System
5. Tracing System
6. Example 1 — `first.cc`: Basic Point-to-Point with UDP Echo
7. Example 2 — `second.cc`: P2P + CSMA (Ethernet LAN)

8. Example 3 — `third.cc`: WiFi + CSMA + P2P Mixed Network
 9. Example 4 — `fourth.cc`: Trace Sources and Callbacks
 10. Example 5 — `fifth.cc`: TCP Congestion Window Tracing
 11. Example 6 — `sixth.cc`: File-Based Tracing (ASCII + PCAP)
 12. Example 7 — `seventh.cc`: Stats Framework + IPv6 Support
 13. The TutorialApp Custom Application
 14. NS-3 Syntax Quick Reference
 15. How to Write Your Own Simulation
 16. Common Mistakes and How to Avoid Them
-

1. What is NS-3?

NS-3 is a **discrete-event network simulator** written in C++. "Discrete-event" means time only advances when something happens (a packet is sent, a timer fires, etc.). There is no continuous real-time clock — the simulator jumps from event to event.

You write a C++ program that **describes a network**, then **run the simulator** which replays all the network events and lets you collect data (logs, packet captures, graphs).

Installation Quick Reference

```
bash

# Clone ns-3.39
git clone https://gitlab.com/nsnam/ns-3-dev.git
cd ns-3-dev
git checkout ns-3.39

# Configure and build
./ns3 configure --enable-examples --enable-tests
./ns3 build

# Test it works
./ns3 run hello-simulator
# Should print: Hello Simulator
```

Running Your Scripts

```
bash
```

```
# Copy a script to scratch (your workspace)
cp examples/tutorial/first.cc scratch/first.cc

# Build (only needed once or after changes)
./ns3 build

# Run
./ns3 run scratch/first

# Run with arguments
./ns3 run "scratch/first --arg1=value1"

# Redirect output to a file
./ns3 run scratch/fifth > scratch/cwnd.dat 2>&1
```

2. Core Concepts

2.1 Node

A **Node** is the fundamental abstraction of a computing device — think of it as a blank computer with no software, no network card, nothing. You add everything to it.

cpp

```
NodeContainer nodes;  
nodes.Create(2);           // Creates 2 nodes, indexed 0 and 1  
nodes.Get(0);              // Returns Ptr<Node> for node 0
```

NodeContainer is just a convenient list of nodes. You can create multiple containers and mix nodes between them:

cpp

```
NodeContainer p2pNodes;  
p2pNodes.Create(2);        // nodes 0 and 1  
  
NodeContainer csmaNodes;  
csmaNodes.Add(p2pNodes.Get(1)); // reuse node 1  
csmaNodes.Create(3);        // add new nodes 2, 3, 4
```

2.2 Application

An **Application** is software that runs on a Node. It generates or consumes traffic.

Built-in applications:

Application	Description
UdpEchoClientApplication	Sends UDP packets and waits for echo
UdpEchoServerApplication	Echoes back every UDP packet it receives
PacketSinkApplication	Receives packets and discards them (traffic sink)
BulkSendApplication	Sends as much TCP data as possible
OnOffApplication	Alternates between sending at a data rate and being silent

You install applications using **Helper** classes:

```
cpp

UdpEchoServerHelper echoServer(9);    // port 9
ApplicationContainer serverApps = echoServer.Install(nodes.Get(1));
serverApps.Start(Seconds(1.0));
serverApps.Stop(Seconds(10.0));
```

2.3 Channel

A **Channel** is the medium over which data travels — the wire, the air, the fiber. It has

physical properties like delay and data rate (depending on the type).

Channel Type	Use Case
PointToPointChannel	Direct link between exactly 2 nodes
CsmaChannel	Ethernet-like shared bus
WifiChannel	Wireless medium

You don't usually create channels directly — the helper does it for you.

2.4 NetDevice

A **NetDevice** is the network interface card (NIC) installed in a node. It connects the node to a channel.

One node can have **multiple NetDevices** (e.g., a router with both Ethernet and WiFi).

NetDevice Type	Use Case
PointToPointNetDevice	One end of a P2P link
CsmaNetDevice	Ethernet NIC
WifiNetDevice	WiFi NIC

```
cpp

NetDeviceContainer devices;
devices = pointToPoint.Install(nodes); // installs P2P devices on both nodes
devices.Get(0);                       // the device on node 0
devices.Get(1);                       // the device on node 1
```

2.5 Topology Helpers

Helpers simplify the process of connecting everything together. They are the "power tools" of NS-3.

```
cpp
```

```
// PointToPointHelper: creates the channel AND the devices
PointToPointHelper pointToPoint;
pointToPoint.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
pointToPoint.SetChannelAttribute("Delay", StringValue("2ms"));
NetDeviceContainer devices = pointToPoint.Install(nodes);

// InternetStackHelper: installs TCP/IP protocol stack on nodes
InternetStackHelper stack;
stack.Install(nodes);

// Ipv4AddressHelper: assigns IP addresses
Ipv4AddressHelper address;
address.SetBase("10.1.1.0", "255.255.255.0"); // network, mask
Ipv4InterfaceContainer interfaces = address.Assign(devices);
```

3. The Simulation Lifecycle

Every NS-3 simulation follows this structure:

SETUP PHASE

Create nodes → Add NetDevices/Channels →
Assign IPs → Install applications →
Configure tracing



SIMULATION PHASE

Simulator::Run()
Events execute in time order until no more events
or Simulator::Stop() time is reached



TEARDOWN PHASE

Simulator::Destroy()
Clean up memory, flush output files

```
// The 3 required lines at the end of every simulation:  
Simulator::Run();           // start running  
Simulator::Destroy();       // clean up  
return 0;
```

When Does the Simulator Stop?

The simulator stops when **one of these** is true:

1. **No more events** in the event queue — all applications have stopped.
2. **A Stop event is scheduled** using `Simulator::Stop(stopTime)`.

⚠ Important: If your network has **recurring events** (like WiFi beacons), the event queue never empties. You **must** call `Simulator::Stop()` for WiFi simulations, or the simulation will run forever.

cpp

```
Simulator::Stop(Seconds(10.0)); // must come BEFORE Simulator::Run()  
Simulator::Run();  
Simulator::Destroy();
```

4. Logging System

NS-3 has a built-in logging framework. You can print messages at different severity levels.

Log Levels (from least to most severe)

Level	Macro	Purpose
<code>LOG_ERROR</code>	<code>NS_LOG_ERROR(msg)</code>	Critical errors
<code>LOG_WARN</code>	<code>NS_LOG_WARN(msg)</code>	Warnings
<code>LOG_DEBUG</code>	<code>NS_LOG_DEBUG(msg)</code>	Debug info
<code>LOG_INFO</code>	<code>NS_LOG_INFO(msg)</code>	Progress info
<code>LOG_FUNCTION</code>	<code>NS_LOG_FUNCTION(this)</code>	Function entry
<code>LOG_LOGIC</code>	<code>NS_LOG_LOGIC(msg)</code>	Logical flow
(special)	<code>NS_LOG_UNCOND(msg)</code>	Always prints, regardless of log level

`LOG_LEVEL_X` enables all levels **at and above** X. So `LOG_LEVEL_INFO` enables ERROR, WARN, DEBUG, and INFO.

Enabling Logs in Code

cpp

```
// At the top of your main() function:  
LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);  
LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
```

Declaring a Log Component in Your Own File

cpp

```
// At the top of your .cc file, outside main():  
NS_LOG_COMPONENT_DEFINE("MySimulationName");  
  
// Then inside functions:  
NS_LOG_INFO("This is an info message");  
NS_LOG_DEBUG("x = " << x);  
NS_LOG_UNCOND("This always prints: " << Simulator::Now().GetSeconds());
```

Enabling Logs from the Shell (Without Changing Code)

bash

```
# Enable all log levels for one component
export NS_LOG=UdpEchoClientApplication=level_all

# Enable + show function names
export 'NS_LOG=UdpEchoClientApplication=level_all|prefix_func'

# Enable for multiple components (colon-separated)
export 'NS_LOG=UdpEchoClientApplication=level_all|prefix_func:UdpEchoServerAppl

# Show simulation time too
export 'NS_LOG=UdpEchoClientApplication=level_all|prefix_func|prefix_time'

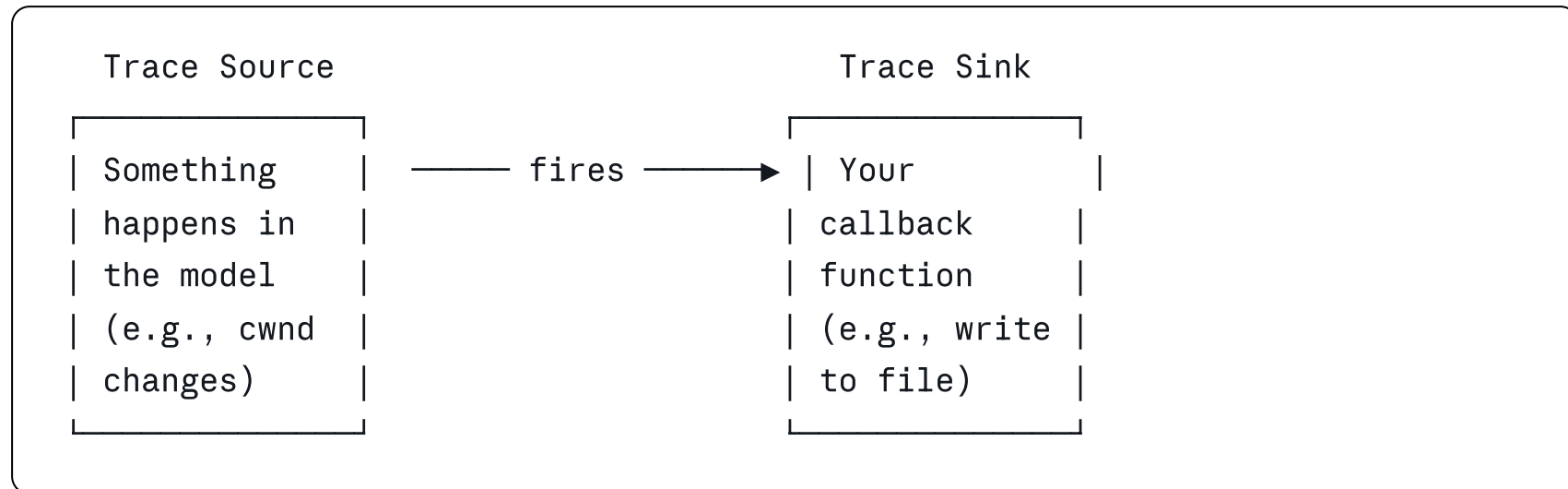
# Enable EVERYTHING (very verbose)
export 'NS_LOG==level_all|prefix_func|prefix_time'
./ns3 run scratch/first > log.out 2>&1

# Turn off logging
export NS_LOG=""
```

5. Tracing System

The logging system is for debugging. The **tracing system** is for collecting simulation data (metrics, graphs, packet captures).

Core Idea: Sources and Sinks



A **trace source** signals when an interesting event happens and provides the data. A **trace sink** is your callback function that receives and records that data.

One trace source can connect to **multiple sinks** simultaneously.

Two Ways to Connect Trace Source to Sink

Method 1: `TraceConnectWithoutContext` — called directly on the object

cpp

```
// Object method – you have a direct pointer to the object
Ptr<MyObject> myObject = CreateObject<MyObject>();
myObject->TraceConnectWithoutContext("TraceSourceName", MakeCallback(&MyFunction));
```

Method 2: `Config::Connect` — uses a path string to find the object

cpp

```
// Path method – useful when you don't have a direct pointer
Config::Connect("/NodeList/7/$ns3::MobilityModel/CourseChange",
                MakeCallback(&CourseChange));

// ConnectWithoutContext – same but callback gets no path prefix
Config::ConnectWithoutContext("/NodeList/*/ $ns3::MobilityModel/CourseChange",
                              MakeCallback(&CourseChange));
```

Writing a Trace Sink (Callback Function)

The function signature must match what the trace source provides. Check the NS-3 Doxygen for each source's signature.

cpp

```

// For a TracedValue<uint32_t> (like CongestionWindow):
// Signature: void f(uint32_t oldValue, uint32_t newValue)
static void
CwndChange(uint32_t oldCwnd, uint32_t newCwnd)
{
    NS_LOG_UNCOND(Simulator::Now().GetSeconds() << "\t" << newCwnd);
}

// For packet drop events:
// Signature: void f(Ptr<const Packet> p)
static void
RxDrop(Ptr<const Packet> p)
{
    NS_LOG_UNCOND("RxDrop at " << Simulator::Now().GetSeconds());
}

```

ASCII Tracing

ASCII tracing records packet events as human-readable text. The prefix characters mean:

Symbol	Meaning
+	Enqueue — packet added to device queue
-	Dequeue — packet removed from device queue
d	Drop — packet dropped (queue full)
r	Receive — packet received by NetDevice

cpp

```
// Enable ASCII trace on all P2P devices
AsciiTraceHelper ascii;
pointToPoint.EnableAsciiAll(ascii.CreateFileStream("trace.tr"));
```

PCAP Tracing

PCAP creates Wireshark-compatible **.pcap** files.

cpp

```
// Enable on all P2P devices (creates second-0-0.pcap, second-1-0.pcap, etc.)
pointToPoint.EnablePcapAll("second");

// Enable on specific device (promiscuous mode = capture all packets on the bus)
csma.EnablePcap("second", csmaDevices.Get(1), true);
```

Read PCAP files:

```
bash

tcpdump -nn -tt -r filename.pcap
# or open in Wireshark
```

6. Example 1 — `first.cc`: Basic Point-to-Point with UDP Echo

What It Does

Two nodes connected by a point-to-point link. Node 0 sends a single UDP packet to node 1, which echoes it back.

```
10.1.1.0
n0 ----- n1
point-to-point
5Mbps, 2ms
```

Full Code Walkthrough

```
cpp

#include "ns3/applications-module.h"
#include "ns3/core-module.h"
#include "ns3/internet-module.h"
#include "ns3/network-module.h"
#include "ns3/point-to-point-module.h"
```

These are NS-3's **module headers**. Each module groups related classes:

- `core-module`: Time, Simulator, logging, callbacks
- `network-module`: Node, Packet, NetDevice, Channel
- `internet-module`: IP, TCP, UDP, routing
- `applications-module`: UdpEcho, BulkSend, OnOff, PacketSink
- `point-to-point-module`: PointToPointHelper, P2P device/channel

cpp

```
using namespace ns3;
```

All NS-3 classes live in the `ns3` namespace. This line avoids typing `ns3::` everywhere.

cpp

```
NS_LOG_COMPONENT_DEFINE("FirstScriptExample");
```

Registers a named log component for this file. Lets you do `export NS_LOG=FirstScriptExample=info` from shell.

cpp

```
int main(int argc, char* argv[])
{
    CommandLine cmd(__FILE__);
    cmd.Parse(argc, argv);
}
```

`CommandLine` parses command-line arguments. Always include `__FILE__` (expands to the filename). Even if you add no custom arguments, this enables built-in NS-3 command-line options like `--PrintHelp`.

cpp

```
Time::SetResolution(Time::NS);
```

Sets the time resolution to nanoseconds. This is the smallest time unit. Usually called once at the start. Optional but good practice.

cpp

```
LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);  
LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
```

Enable INFO-level logging for the built-in echo applications. This causes them to print messages when they send/receive packets.

cpp

```
NodeContainer nodes;  
nodes.Create(2);
```

Creates 2 nodes (node 0 and node 1). Simple!

cpp

```
PointToPointHelper pointToPoint;  
pointToPoint.SetDeviceAttribute("DataRate", StringValue("5Mbps"));  
pointToPoint.SetChannelAttribute("Delay", StringValue("2ms"));
```

Configure the P2P helper. `SetDeviceAttribute` sets properties on the NetDevice (bandwidth), `SetChannelAttribute` sets properties on the Channel (propagation delay). These use NS-3's attribute system — values are passed as `StringValue`, `UintegerValue`, `DoubleValue`, etc.

cpp

```
NetDeviceContainer devices;  
devices = pointToPoint.Install(nodes);
```

This single line: creates the P2P channel, creates two P2P NetDevices, installs one on each node, and connects them. Returns a container of the two devices.

cpp

```
InternetStackHelper stack;  
stack.Install(nodes);
```

Installs the full TCP/IP stack (IPv4, IPv6, TCP, UDP, ARP, routing) on both nodes. Without this, nodes can't use IP.

cpp

```
Ipv4AddressHelper address;  
address.SetBase("10.1.1.0", "255.255.255.0");  
Ipv4InterfaceContainer interfaces = address.Assign(devices);
```

Assigns IP addresses starting from 10.1.1.1 (the .0 is the network address). Node 0 gets 10.1.1.1, node 1 gets 10.1.1.2. Returns an Ipv4InterfaceContainer to look up addresses later.

cpp

```
UdpEchoServerHelper echoServer(9); // port 9  
ApplicationContainer serverApps = echoServer.Install(nodes.Get(1));  
serverApps.Start(Seconds(1.0));  
serverApps.Stop(Seconds(10.0));
```

Install a UDP echo server on node 1 listening on port 9. It starts at simulation time 1s and stops at 10s.

cpp

```
UdpEchoClientHelper echoClient(interfaces.GetAddress(1), 9);
echoClient.SetAttribute("MaxPackets", UIntegerValue(1));
echoClient.SetAttribute("Interval", TimeValue(Seconds(1.0)));
echoClient.SetAttribute("PacketSize", UIntegerValue(1024));
ApplicationContainer clientApps = echoClient.Install(nodes.Get(0));
clientApps.Start(Seconds(2.0));
clientApps.Stop(Seconds(10.0));
```

Install a UDP echo client on node 0. `interfaces.GetAddress(1)` gets node 1's IP address. It sends 1 packet of 1024 bytes. The client starts at 2s (after the server at 1s).

cpp

```
Simulator::Run();
Simulator::Destroy();
return 0;
}
```

Run the simulation, clean up, done.

Key Takeaway

The general pattern for every simulation:

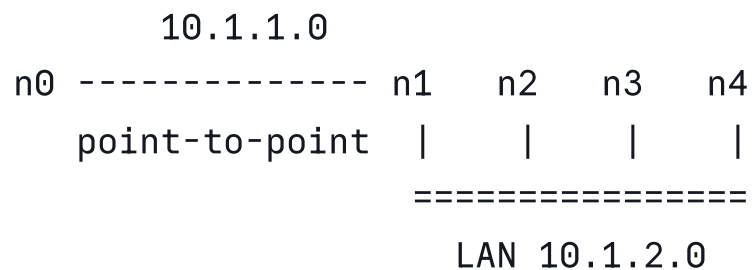
1. Create nodes

2. Create devices + channels (via helpers)
 3. Install IP stack
 4. Assign IP addresses
 5. Install applications
 6. Set start/stop times
 7. Run
-

7. Example 2 — `second.cc`: P2P + CSMA (Ethernet LAN)

What It Does

Node 0 connects to node 1 via point-to-point. Node 1 also connects to nodes 2, 3, 4 via a CSMA (Ethernet) LAN. Node 0 sends a UDP packet to node 4 on the LAN.



Key New Concepts

Node reuse across containers:

```
cpp

NodeContainer p2pNodes;
p2pNodes.Create(2);           // nodes 0, 1

NodeContainer csmaNodes;
csmaNodes.Add(p2pNodes.Get(1)); // node 1 is SHARED — it appears in BOTH containers
csmaNodes.Create(nCsma);       // creates nodes 2, 3, 4
```

Node 1 is a **router** — it has one P2P interface (10.1.1.2) and one CSMA interface (10.1.2.1).

CSMA Helper:

```
cpp

CsmaHelper csma;
csma.SetChannelAttribute("DataRate", StringValue("100Mbps"));
csma.SetChannelAttribute("Delay", TimeValue(NanoSeconds(6560)));
NetDeviceContainer csmaDevices = csma.Install(csmaNodes);
```

Note: CSMA channel delay is set in `NanoSeconds()`, not as a string. Multiple ways to specify time values.

Partial stack installation (avoid double-installing):

```
cpp

// Node 1 is in csmaNodes, so we use p2pNodes.Get(0) for node 0 separately
stack.Install(p2pNodes.Get(0));
stack.Install(csmaNodes);    // installs on nodes 1, 2, 3, 4
// If you called stack.Install(p2pNodes), node 1 would get installed twice – ER
```

Separate IP subnets:

```
cpp

address.SetBase("10.1.1.0", "255.255.255.0");
p2pInterfaces = address.Assign(p2pDevices);    // 10.1.1.1, 10.1.1.2

address.SetBase("10.1.2.0", "255.255.255.0"); // different subnet!
csmaInterfaces = address.Assign(csmaDevices);  // 10.1.2.1, .2, .3, .4
```

Routing:

```
cpp
```

```
Ipv4GlobalRoutingHelper::PopulateRoutingTables();
```

This automatically calculates and installs routing tables on all nodes so they know how to forward packets. Without this, node 0 wouldn't know how to reach the 10.1.2.x subnet.

PCAP tracing:

```
cpp
pointToPoint.EnablePcapAll("second");           // all P2P devices
csma.EnablePcap("second", csmaDevices.Get(1), true); // one CSMA device, promiscuous
```

Promiscuous mode (`true`) means the device captures **all** packets on the bus, not just its own.

Command Line Arguments

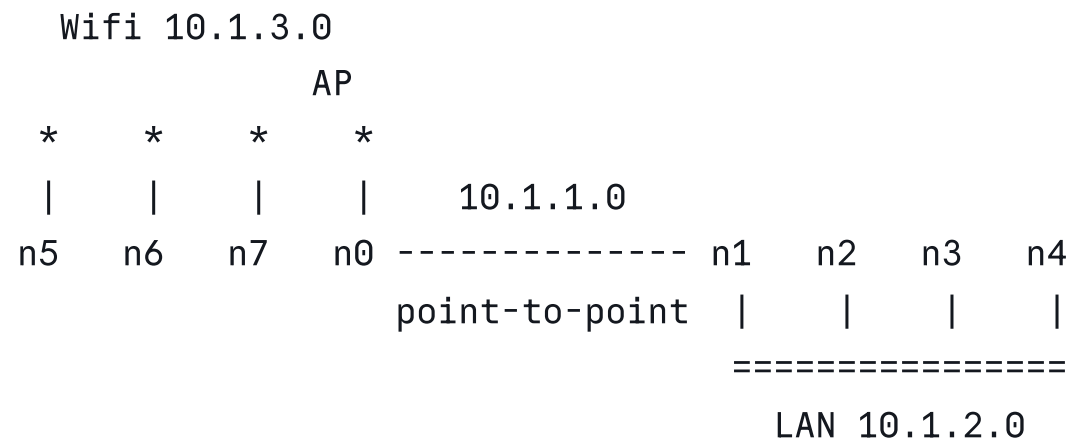
```
cpp
uint32_t nCsma = 3;
CommandLine cmd(__FILE__);
cmd.AddValue("nCsma", "Number of CSMA nodes", nCsma);
cmd.Parse(argc, argv);
```

Now you can run: `./ns3 run "scratch/second --nCsma=5"` to change the number of CSMA nodes at runtime without editing code.

8. Example 3 — `third.cc`: WiFi + CSMA + P2P Mixed Network

What It Does

The most complex topology so far — three different network types connected together.



Node 0 is both the WiFi Access Point and one end of the P2P link.

WiFi Setup (New Concepts)

```
cpp
```

```
// 1. Create WiFi channel (the air)
YansWifiChannelHelper channel = YansWifiChannelHelper::Default();
YansWifiPhyHelper phy;
phy.SetChannel(channel.Create());
```

`YansWifiChannelHelper::Default()` creates a standard WiFi channel with log-distance path loss and constant speed propagation.

```
cpp

// 2. Set up MAC layer
WifiMacHelper mac;
Ssid ssid = Ssid("ns-3-ssid");    // network name (like WiFi SSID)
WifiHelper wifi;
```

```
cpp

// 3. Install WiFi on Station nodes (the clients)
mac.SetType("ns3::StaWifiMac",
            "Ssid", SsidValue(ssid),
            "ActiveProbing", BooleanValue(false)); // passive scanning
NetDeviceContainer staDevices = wifi.Install(phy, mac, wifiStaNodes);
```

cpp

```
// 4. Install WiFi on Access Point node  
mac.SetType("ns3::ApWifiMac", "Ssid", SsidValue(ssid));  
NetDeviceContainer apDevices = wifi.Install(phy, mac, wifiApNode);
```

Mobility Model (Required for WiFi)

WiFi requires nodes to have a position in space. Without a mobility model, nodes have no location and can't communicate.

cpp

```
MobilityHelper mobility;

// Grid layout for initial positions
mobility.SetPositionAllocator("ns3::GridPositionAllocator",
                              "MinX", DoubleValue(0.0),
                              "MinY", DoubleValue(0.0),
                              "DeltaX", DoubleValue(5.0), // 5m between nodes
                              "DeltaY", DoubleValue(10.0),
                              "GridWidth", UIntegerValue(3),
                              "LayoutType", StringValue("RowFirst"));

// WiFi stations move randomly within a bounding box
mobility.SetMobilityModel("ns3::RandomWalk2dMobilityModel",
                          "Bounds", RectangleValue(Rectangle(-50, 50, -50, 50))
mobility.Install(wifiStaNodes);

// Access Point stays fixed
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(wifiApNode);
```

Why Simulator::Stop() is Mandatory Here

WiFi APs continuously broadcast beacon frames. This means the event queue never

becomes empty — there's always a "next beacon" event scheduled. Without `Simulator::Stop()`, the simulation runs forever.

cpp

```
Simulator::Stop(Seconds(10.0)); // REQUIRED for WiFi
Simulator::Run();
```

Tracing for WiFi

cpp

```
if (tracing)
{
    phy.SetPcapDataLinkType(WifiPhyHelper::DLT_IEEE802_11_RADIO);
    pointToPoint.EnablePcapAll("third");
    phy.EnablePcap("third", apDevices.Get(0)); // WiFi AP pcap
    csma.EnablePcap("third", csmaDevices.Get(0), true);
}
```

9. Example 4 — `fourth.cc`: Trace Sources and Callbacks

What It Does

This is a teaching example — no real network. It shows how to create your own traced variable and connect a callback to it.

Defining a Traced Object

```
cpp
```

```

class MyObject : public Object
{
public:
    static TypeId GetTypeId()
    {
        static TypeId tid = TypeId("MyObject")
            .SetParent<Object>()
            .SetGroupName("Tutorial")
            .AddConstructor<MyObject>()
            .AddTraceSource(
                "MyInteger",           // name
                "An integer value to trace.", // description
                MakeTraceSourceAccessor(&MyObject::m_myInt), // which variable
                "ns3::TracedValueCallback::Int32" // callback signature
            );
        return tid;
    }

    TracedValue<int32_t> m_myInt; // The special traced variable
};

```

TracedValue<T> is a wrapper around a normal value (int, double, etc.) that fires a callback whenever the value changes. It behaves exactly like the underlying type for read/write

operations, but notifies all connected sinks on every assignment.

The Callback Function

```
cpp

void IntTrace(int32_t oldValue, int32_t newValue)
{
    std::cout << "Traced " << oldValue << " to " << newValue << std::endl;
}
```

The signature must match the `TracedValueCallback::Int32` typedef, which is: `void (int32_t, int32_t)` — old value, new value.

Connecting and Triggering

```
cpp
```

```

int main(int argc, char* argv[])
{
    Ptr<MyObject> myObject = CreateObject<MyObject>();

    // Connect the trace sink to the trace source named "MyInteger"
    myObject->TraceConnectWithoutContext("MyInteger", MakeCallback(&IntTrace));

    // This assignment triggers the callback automatically!
    myObject->m_myInt = 1234;
    // Output: "Traced 0 to 1234"

    return 0;
}

```

`Ptr<T>` is NS-3's smart pointer (like `std::shared_ptr`). Always use `CreateObject<T>()` to create NS-3 objects — never `new T()`.

`MakeCallback(&function)` wraps a function pointer into NS-3's callback system.

How TracedValue Works Internally

When you write `myObject->m_myInt = 1234`, the `TracedValue` wrapper:

1. Records the old value (0)

2. Sets the new value (1234)
3. Calls all connected callbacks with (old=0, new=1234)

This is what makes NS-3 tracing elegant — you write to a variable normally, and the instrumentation happens automatically.

10. Example 5 — `fifth.cc`: TCP Congestion Window Tracing

What It Does

Two nodes connected by P2P. Node 0 sends TCP data to node 1. We observe how the TCP congestion window (`cwnd`) changes over time. An error model introduces random packet drops to trigger congestion events.

New Concepts

TCP Configuration:

```
cpp
```

```
// Set the congestion control algorithm
Config::SetDefault("ns3::TcpL4Protocol::SocketType", StringValue("ns3::TcpNewRe
// Set initial window size (1 packet for slow start demo)
Config::SetDefault("ns3::TcpSocket::InitialCwnd", UIntegerValue(1));
// Set recovery algorithm
Config::SetDefault("ns3::TcpL4Protocol::RecoveryType",
                  TypeIdValue(TypeId::LookupByName("ns3::TcpClassicRecovery"))
```

`Config::SetDefault` changes the **default attribute values** for all instances of a class created after this call. This is the global configuration approach.

Error Model (packet drops):

```
cpp

Ptr<RateErrorModel> em = CreateObject<RateErrorModel>();
em->SetAttribute("ErrorRate", DoubleValue(0.00001)); // 0.001% drop rate
devices.Get(1)->SetAttribute("ReceiveErrorModel", PointerValue(em));
```

This tells node 1's NetDevice to randomly drop received packets at the given rate, simulating network errors.

PacketSink (TCP receiver):

cpp

```
uint16_t sinkPort = 8080;  
Address sinkAddress(InetSocketAddress(interfaces.GetAddress(1), sinkPort));  
PacketSinkHelper packetSinkHelper("ns3::TcpSocketFactory",  
                                   InetSocketAddress(Ipv4Address::GetAny(), sinkPort),  
                                   sinkAddress);  
ApplicationContainer sinkApps = packetSinkHelper.Install(nodes.Get(1));  
sinkApps.Start(Seconds(0.));  
sinkApps.Stop(Seconds(20.));
```

`PacketSink` is a passive receiver — it accepts connections and just discards incoming data.

`Ipv4Address::GetAny()` means "listen on all interfaces". `"ns3::TcpSocketFactory"` means use TCP.

Creating a raw socket and connecting a trace:

cpp

```
// Create a TCP socket manually on node 0  
Ptr<Socket> ns3TcpSocket = Socket::CreateSocket(nodes.Get(0), TcpSocketFactory::GetDefault());  
  
// Connect a callback to the CongestionWindow trace source on this socket  
ns3TcpSocket->TraceConnectWithoutContext("CongestionWindow", MakeCallback(&CwndUpdate, this));
```

This is the key pattern: create the socket first, hook the trace, then pass the socket to the application.

Installing the custom TutorialApp:

```
cpp

Ptr<TutorialApp> app = CreateObject<TutorialApp>();
app->Setup(ns3TcpSocket, sinkAddress, 1040, 1000, DataRate("1Mbps"));
//          ^dest addr  ^pkt  ^n    ^rate
//          size  pkts
nodes.Get(0)->AddApplication(app);
app->SetStartTime(Seconds(1.));
app->SetStopTime(Seconds(20.));
```

Tracing packet drops on the physical layer:

```
cpp

devices.Get(1)->TraceConnectWithoutContext("PhyRxDrop", MakeCallback(&RxDrop));
```

`PhyRxDrop` is a trace source on NetDevice that fires when the receive error model drops a packet.

The Callback Functions

cpp

```
static void
CwndChange(uint32_t oldCwnd, uint32_t newCwnd)
{
    // Simulator::Now().GetSeconds() – current simulation time as a double
    NS_LOG_UNCOND(Simulator::Now().GetSeconds() << "\t" << newCwnd);
}

static void
RxDrop(Ptr<const Packet> p)
{
    NS_LOG_UNCOND("RxDrop at " << Simulator::Now().GetSeconds());
}
```

Running and Plotting

bash

```
# Run and capture output
./ns3 run fifth > scratch/cwnd.dat 2>&1

# Plot with gnuplot
gnuplot
gnuplot> set terminal png size 640,480
gnuplot> set output "cwnd.png"
gnuplot> plot "cwnd.dat" using 1:2 title 'Congestion Window' with linespoints
gnuplot> exit
```

The output has two columns: simulation time (col 1) and cwnd in bytes (col 2).

11. Example 6 — `sixth.cc`: File-Based Tracing (ASCII + PCAP)

What It Does

Same topology as `fifth.cc`, but instead of printing to stdout, all trace data is written to files.

Key Differences from `fifth.cc`

Writing congestion window data to a file:

cpp

```
AsciiTraceHelper asciiTraceHelper;  
Ptr<OutputStreamWrapper> stream = asciiTraceHelper.CreateFileStream("sixth.cwnd  
ns3TcpSocket->TraceConnectWithoutContext("CongestionWindow",  
MakeBoundCallback(&CwndChange, stream))
```

`MakeBoundCallback` is like `MakeCallback` but **binds extra arguments** to the front of the callback. Here, `stream` is passed as the first argument to `CwndChange` every time it fires.

The updated callback signature (now takes the stream):

cpp

```
static void  
CwndChange(Ptr<OutputStreamWrapper> stream, uint32_t oldCwnd, uint32_t newCwnd)  
{  
    NS_LOG_UNCOND(Simulator::Now().GetSeconds() << "\t" << newCwnd); // still  
    *stream->GetStream() << Simulator::Now().GetSeconds() << "\t" << oldCwnd <<  
        << std::endl; // AND writes to file  
}
```

`stream->GetStream()` returns a `std::ostream*`. Dereference with `*` and use `<<` just like `cout`.

Writing dropped packets to a PCAP file:

```
cpp

PcapHelper pcapHelper;
Ptr<PcapFileWrapper> file =
    pcapHelper.CreateFile("sixth.pcap", std::ios::out, PcapHelper::DLT_PPP);
devices.Get(1)->TraceConnectWithoutContext("PhyRxDrop", MakeBoundCallback(&RxDr
```

```
cpp

static void
RxDrop(Ptr<PcapFileWrapper> file, Ptr<const Packet> p)
{
    NS_LOG_UNCOND("RxDrop at " << Simulator::Now().GetSeconds());
    file->Write(Simulator::Now(), p);    // write packet + timestamp to pcap
}
```

MakeBoundCallback vs **MakeCallback**

```
cpp
```

```
// MakeCallback – function called with exactly the arguments the trace source p
MakeCallback(&CwndChange)
// CwndChange(uint32_t old, uint32_t new) – 2 args from trace source

// MakeBoundCallback – prepends extra bound arguments
MakeBoundCallback(&CwndChange, stream)
// CwndChange(Ptr<OutputStreamWrapper> stream, uint32_t old, uint32_t new)
// The stream is bound once at setup; only old+new come from the trace source
```

12. Example 7 — `seventh.cc`: Stats Framework + IPv6 Support

What It Does

Extends `sixth.cc` with:

1. The NS-3 **Stats framework** (`GnuplotHelper`, `FileHelper`) for automatic data collection
2. **IPv6 support** via a command-line flag

IPv6 Support

cpp

```
bool useV6 = false;  
CommandLine cmd(__FILE__);  
cmd.AddValue("useIpv6", "Use Ipv6", useV6);  
cmd.Parse(argc, argv);
```

cpp

```

if (!useV6)
{
    Ipv4AddressHelper address;
    address.SetBase("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer interfaces = address.Assign(devices);
    sinkAddress = InetSocketAddress(interfaces.GetAddress(1), sinkPort);
    anyAddress = InetSocketAddress(Ipv4Address::GetAny(), sinkPort);
    probeType = "ns3::Ipv4PacketProbe";
    tracePath = "/NodeList/*/ns3::Ipv4L3Protocol/Tx";
}
else
{
    Ipv6AddressHelper address;
    address.SetBase("2001:0000:f00d:cafe::", Ipv6Prefix(64));
    Ipv6InterfaceContainer interfaces = address.Assign(devices);
    sinkAddress = Inet6SocketAddress(interfaces.GetAddress(1, 1), sinkPort);
    anyAddress = Inet6SocketAddress(Ipv6Address::GetAny(), sinkPort);
    probeType = "ns3::Ipv6PacketProbe";
    tracePath = "/NodeList/*/ns3::Ipv6L3Protocol/Tx";
}

```

Run with IPv6: `./ns3 run "scratch/seventh --useIpv6=1"`

GnuplotHelper — Automatic Plot Generation

cpp

```
GnuplotHelper plotHelper;

// Configure output file name and axis labels
plotHelper.ConfigurePlot("seventh-packet-byte-count",
                        "Packet Byte Count vs. Time",
                        "Time (Seconds)",
                        "Packet Byte Count");

// Specify what to plot:
// - probeType: what kind of probe to use
// - tracePath: where in the namespace to connect
// - "OutputBytes": which output of the probe to use
// - label: legend label
// - key position
plotHelper.PlotProbe(probeType,
                    tracePath,
                    "OutputBytes",
                    "Packet Byte Count",
                    GnuplotAggregator::KEY_BELOW);
```

This automatically generates `.plt` and `.dat` files and a `.png` when combined with gnuplot.

FileHelper — Automatic File Writing

```
cpp

FileHelper fileHelper;
fileHelper.ConfigureFile("seventh-packet-byte-count", FileAggregator::FORMATTED);
fileHelper.Set2dFormat("Time (Seconds) = %.3e\tPacket Byte Count = %.0f");
fileHelper.WriteProbe(probeType, tracePath, "OutputBytes");
```

This automatically collects data and writes it to a formatted text file — no manual callback needed.

13. The TutorialApp Custom Application

The `TutorialApp` class (in `tutorial-app.h` and `tutorial-app.cc`) is a minimal custom TCP sending application created for these examples. Understanding it teaches you how to write your own applications.

Class Structure

```
cpp
```

```

class TutorialApp : public Application    // must inherit from Application
{
public:
    static TypeId GetTypeId();            // required for all NS-3 objects
    void Setup(Ptr<Socket> socket, Address address,
               uint32_t packetSize, uint32_t nPackets, DataRate dataRate);

private:
    void StartApplication() override;     // called when app start time arrives
    void StopApplication() override;      // called when app stop time arrives
    void ScheduleTx();                     // schedule the next send event
    void SendPacket();                     // actually send one packet

    Ptr<Socket> m_socket;
    Address m_peer;
    uint32_t m_packetSize;
    uint32_t m_nPackets;
    DataRate m_dataRate;
    EventId m_sendEvent;    // handle to the scheduled send event
    bool m_running;
    uint32_t m_packetsSent;
};

```

TypeId Registration (required for every NS-3 object class)

```
cpp

TypeId TutorialApp::GetTypeId()
{
    static TypeId tid = TypeId("TutorialApp")
        .SetParent<Application>()
        .SetGroupName("Tutorial")
        .AddConstructor<TutorialApp>();
    return tid;
}
```

Every class that inherits from `Object` must implement `GetTypeId()`. This registers the class with NS-3's type system, enabling features like `CreateObject<T>()`, attribute system, and tracing.

StartApplication and StopApplication

```
cpp
```

```

void TutorialApp::StartApplication()
{
    m_running = true;
    m_packetsSent = 0;
    m_socket->Bind();           // bind to any local port
    m_socket->Connect(m_peer); // TCP 3-way handshake
    SendPacket();              // send first packet
}

void TutorialApp::StopApplication()
{
    m_running = false;
    if (m_sendEvent.IsRunning())
        Simulator::Cancel(m_sendEvent); // cancel any pending send
    if (m_socket)
        m_socket->Close(); // TCP FIN/RST
}

```

`StartApplication()` is called automatically by the simulator when the application's start time arrives. Same for `StopApplication()`.

Sending Packets with Scheduling

cpp

```

void TutorialApp::SendPacket()
{
    Ptr<Packet> packet = Create<Packet>(m_packetSize); // create empty packet
    m_socket->Send(packet);                             // hand to TCP

    if (++m_packetsSent < m_nPackets)
        ScheduleTx(); // if more to send, schedule next one
}

void TutorialApp::ScheduleTx()
{
    if (m_running)
    {
        // Calculate time for one packet at current data rate
        // time = bits / bits_per_second
        Time tNext(Seconds(m_packetSize * 8 / static_cast<double>(m_dataRate.Get
m_sendEvent = Simulator::Schedule(tNext, &TutorialApp::SendPacket, this
    }
}

```

`Simulator::Schedule(delay, &function, args...)` schedules a function call to happen `delay` time from now. Returns an `EventId` that can be used to cancel the event.

14. NS-3 Syntax Quick Reference

Time Values

cpp

```
Seconds(1.0)           // 1 second
MilliSeconds(500)      // 500 ms
MicroSeconds(100)      // 100  $\mu$ s
NanoSeconds(6560)      // 6560 ns

Simulator::Now()        // current simulation time as Time object
Simulator::Now().GetSeconds() // as double (seconds)
Simulator::Now().GetMilliSeconds() // as int64 (milliseconds)
```

Data Rate Values

cpp

```
DataRate("5Mbps")      // 5 megabits per second
DataRate("100Mbps")
DataRate("1Gbps")
DataRate("56Kbps")
```

Address Types

cpp

```
// IPv4 socket address (IP + port)
InetSocketAddress(Ipv4Address("10.1.1.2"), 8080)
InetSocketAddress(interfaces.GetAddress(1), 8080)
InetSocketAddress(Ipv4Address::GetAny(), 8080) // wildcard

// IPv6 socket address
Inet6SocketAddress(Ipv6Address("2001::1"), 8080)
Inet6SocketAddress(Ipv6Address::GetAny(), 8080)
```

Pointer Types

cpp

```
Ptr<Node> node = nodes.Get(0); // smart pointer
Ptr<Socket> socket = Socket::CreateSocket(node, TcpSocketFactory::GetTypeId());
Ptr<MyClass> obj = CreateObject<MyClass>(); // always use this, never new
```

Attribute System

cpp

```
// String values
StringValue("5Mbps")
StringValue("2ms")

// Numeric values
UIntegerValue(1024)
DoubleValue(0.00001)
BooleanValue(true)

// Time values
TimeValue(Seconds(1.0))
TimeValue(NanoSeconds(6560))

// Type ID values
TypeIdValue(TypeId::LookupByName("ns3::TcpClassicRecovery"))

// Pointer values
PointerValue(em) // where em is a Ptr<SomeObject>
```

Scheduling Events

cpp

```
// Schedule at absolute time
Simulator::Schedule(Seconds(5.0), &MyFunction, arg1, arg2);

// Schedule relative to now (use in callbacks/apps)
Time delay = Seconds(0.1);
EventId event = Simulator::Schedule(delay, &TutorialApp::SendPacket, this);

// Cancel a scheduled event
Simulator::Cancel(event);
event.IsRunning(); // returns true if not yet executed and not cancelled
```

Common Container Methods

cpp

```
NodeContainer nodes;  
nodes.Create(n);           // create n nodes  
nodes.Add(other);          // add nodes from another container  
nodes.Get(i);              // get Ptr<Node> at index i  
nodes.GetN();              // number of nodes  
  
NetDeviceContainer devices;  
devices.Get(i);            // get Ptr<NetDevice> at index i  
  
ApplicationContainer apps;  
apps.Start(Seconds(1.0));  // start all apps at t=1  
apps.Stop(Seconds(10.0));  // stop all apps at t=10  
apps.Get(i);              // get Ptr<Application> at index i
```

15. How to Write Your Own Simulation

Follow this step-by-step template:

Step 1: Decide Your Topology and Protocol

Before writing code, answer:

- How many nodes?

- How are they connected? (P2P, CSMA, WiFi, or combination)
- What protocol? (UDP, TCP)
- What traffic pattern? (Echo, bulk transfer, on-off)
- What do you want to measure? (Throughput, delay, packet loss, cwnd)

Step 2: Start from the Template

cpp

```
#include "ns3/applications-module.h"
#include "ns3/core-module.h"
#include "ns3/internet-module.h"
#include "ns3/network-module.h"
#include "ns3/point-to-point-module.h"
// Add csma-module, wifi-module, mobility-module, stats-module as needed

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("MySimulation");

int main(int argc, char* argv[])
{
    // ---- PARAMETERS (with defaults) ----
    uint32_t nNodes = 2;
    double simTime = 10.0;
    std::string dataRate = "5Mbps";

    CommandLine cmd(__FILE__);
    cmd.AddValue("nNodes", "Number of nodes", nNodes);
    cmd.AddValue("simTime", "Simulation time", simTime);
    cmd.Parse(argc, argv);

    // ---- NODES ----
```

```
NodeContainer nodes;
nodes.Create(nNodes);

// ---- DEVICES + CHANNELS ----
PointToPointHelper p2p;
p2p.SetDeviceAttribute("DataRate", StringValue(dataRate));
p2p.SetChannelAttribute("Delay", StringValue("2ms"));
NetDeviceContainer devices = p2p.Install(nodes);

// ---- INTERNET STACK ----
InternetStackHelper stack;
stack.Install(nodes);

// ---- IP ADDRESSES ----
Ipv4AddressHelper address;
address.SetBase("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign(devices);

// ---- ROUTING (for multi-hop) ----
// Ipv4GlobalRoutingHelper::PopulateRoutingTables();

// ---- APPLICATIONS ----
uint16_t port = 9;
UdpEchoServerHelper server(port);
```

```
ApplicationContainer serverApps = server.Install(nodes.Get(1));
serverApps.Start(Seconds(1.0));
serverApps.Stop(Seconds(simTime));

UdpEchoClientHelper client(interfaces.GetAddress(1), port);
client.SetAttribute("MaxPackets", UintegerValue(10));
client.SetAttribute("Interval", TimeValue(Seconds(1.0)));
client.SetAttribute("PacketSize", UintegerValue(1024));
ApplicationContainer clientApps = client.Install(nodes.Get(0));
clientApps.Start(Seconds(2.0));
clientApps.Stop(Seconds(simTime));

// ---- TRACING ----
// AsciiTraceHelper ascii;
// p2p.EnableAsciiAll(ascii.CreateFileStream("my-sim.tr"));
// p2p.EnablePcapAll("my-sim");

// ---- RUN ----
// Simulator::Stop(Seconds(simTime)); // required for WiFi
Simulator::Run();
Simulator::Destroy();
return 0;
}
```

Step 3: Add Your Network Type

For CSMA (Ethernet):

```
cpp

#include "ns3/csma-module.h"

CsmaHelper csma;
csma.SetChannelAttribute("DataRate", StringValue("100Mbps"));
csma.SetChannelAttribute("Delay", TimeValue(NanoSeconds(6560)));
NetDeviceContainer csmaDevices = csma.Install(csmaNodes);
```

For WiFi:

```
cpp
```

```
#include "ns3/yans-wifi-helper.h"
#include "ns3/mobility-module.h"
#include "ns3/ssid.h"

YansWifiChannelHelper channel = YansWifiChannelHelper::Default();
YansWifiPhyHelper phy;
phy.SetChannel(channel.Create());

WifiMacHelper mac;
WifiHelper wifi;
Ssid ssid = Ssid("my-network");

mac.SetType("ns3::StaWifiMac", "Ssid", SsidValue(ssid), "ActiveProbing", Boolean
NetDeviceContainer staDevices = wifi.Install(phy, mac, staNodes);
mac.SetType("ns3::ApWifiMac", "Ssid", SsidValue(ssid));
NetDeviceContainer apDevices = wifi.Install(phy, mac, apNode);

MobilityHelper mobility;
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(allNodes);

Simulator::Stop(Seconds(simTime)); // REQUIRED for WiFi
```

Step 4: Add Tracing

Simple console output:

```
cpp  
  
NS_LOG_UNCOND("My value: " << someVar);
```

Write to file:

```
cpp  
  
AsciiTraceHelper asciiTraceHelper;  
Ptr<OutputStreamWrapper> stream = asciiTraceHelper.CreateFileStream("output.dat"  
// In your callback:  
*stream->GetStream() << time << "\t" << value << std::endl;
```

PCAP capture:

```
cpp  
  
p2p.EnablePcapAll("my-simulation");
```

Hook a trace source:

cpp

```
// Find trace source in Doxygen, note its callback signature
// Write your callback function with matching signature
// Connect:
someObject->TraceConnectWithoutContext("TraceName", MakeCallback(&MyCallback));
// Or with extra data:
someObject->TraceConnectWithoutContext("TraceName", MakeBoundCallback(&MyCallba
```

16. Common Mistakes and How to Avoid Them

1. Forgetting `Simulator::Stop()` for WiFi

Problem: Simulation runs forever. **Fix:** Always add `Simulator::Stop(Seconds(N))` before `Simulator::Run()` when using WiFi.

2. Installing Internet Stack Twice on Same Node

Problem: Crash or weird behavior when a node appears in two NodeContainers. **Fix:** Install the stack carefully — only once per node.

cpp

```
// BAD:
stack.Install(p2pNodes); // installs on node 0 and 1
stack.Install(csmaNodes); // installs on node 1 AGAIN (node 1 is shared) – CRA

// GOOD:
stack.Install(p2pNodes.Get(0)); // just node 0
stack.Install(csmaNodes); // nodes 1, 2, 3, 4
```

3. Missing Routing for Multi-Hop Networks

Problem: Packets don't arrive at destination across subnets. **Fix:** Call

`Ipv4GlobalRoutingHelper::PopulateRoutingTables()` after all IP addresses are assigned.

4. Starting Client Before Server

Problem: Client sends to an address with no listening socket — connection refused or packet dropped. **Fix:** Always start the server at least a few milliseconds before the client.

```
cpp

serverApps.Start(Seconds(1.0)); // server first
clientApps.Start(Seconds(2.0)); // client after
```

5. Wrong Callback Signature

Problem: Compile error or crash when connecting to a trace source. **Fix:** Look up the trace source in NS-3 Doxygen. Under "Trace Sources", find the `TracedCallback` typedef for the source. Your callback function's parameters must match exactly.

6. Using `new` Instead of `CreateObject`

Problem: NS-3's reference counting and type system won't work. **Fix:** Always use `CreateObject<T>()` for NS-3 objects.

```
cpp

// BAD:
MyObject* obj = new MyObject();

// GOOD:
Ptr<MyObject> obj = CreateObject<MyObject>();
```

7. Not Calling `Simulator::Destroy()` After `Run()`

Problem: Resources not cleaned up, output file buffers not flushed. **Fix:** Always call `Simulator::Destroy()` at the end.

8. IP Address Subnet Conflicts

Problem: Two network segments using the same subnet → routing fails. **Fix:** Give each segment its own unique subnet.

cpp

```
address.SetBase("10.1.1.0", "255.255.255.0"); // P2P link
address.SetBase("10.1.2.0", "255.255.255.0"); // CSMA LAN
address.SetBase("10.1.3.0", "255.255.255.0"); // WiFi network
```

Summary: The NS-3 Mental Model

PHYSICAL WORLD

Computer →
Network cable →
Network card →
Software app →
TCP/IP stack →

NS-3 ABSTRACTION

Node
Channel
NetDevice
Application
InternetStack

Interesting event → Trace Source (fires callback)
Measurement code → Trace Sink (your callback function)
Event in time → Simulator::Schedule(...)

The simulation is just a C++ program that builds a virtual network and lets the simulator "run time forward," firing events in chronological order. Everything that happens — packets sent, queues filled, congestion detected — is a scheduled event, and you can hook into any of those events through the tracing system.

Prepared as a study guide for CSE 322 — Computer Networking Sessional. Based on NS-3 v3.39 tutorial examples.